Kubernetes

What is Kubernetes?

Kubernetes is an open-source container orchestration platform used to:

- Deploy applications
- Scale applications
- Manage containers automatically

Simple definition:

Kubernetes is a tool used to automate deployment, scaling, and management of containerized applications.

Kubernetes Architecture

Master Node / Control Plane Components

1. API Server

- Main entry point of Kubernetes
- Accepts all kubectl commands
- Communicates with all components

Simple:

API Server is the brain gateway of Kubernetes.

---

2. ETCD

- Key-value database
- Stores cluster information
- Stores pod details, secrets, configs

Simple:

ETCD stores all Kubernetes cluster data.

---

3. Scheduler

- Decides which node should run the pod

Simple:

Scheduler assigns pods to nodes.

---

4. Controller Manager

- Monitors cluster state
- Ensures desired state is maintained

Example:

- If pod crashes → creates new pod

Simple:

Controller Manager maintains desired state.

Worker Node Components

1. Kubelet

- Agent running on every node
- Talks with API server
- Creates pods

Simple:

Kubelet manages pods on nodes.

---

2. Kube Proxy

- Handles networking
- Maintains communication between services and pods

Simple:

Kube Proxy manages network rules.

---

3. Container Runtime

Examples:

- Docker
- containerd
- CRI-O

Simple:

Container runtime runs containers.

Kubernetes Objects

1. Pod

Smallest deployable unit in Kubernetes.

Contains:

- One or more containers

Simple:

Pod is a wrapper around containers.

Pod YAML

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
```

```
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
```

Create Pod

```
kubectl apply -f pod.yml
```

ReplicaSet

Maintains fixed number of pod replicas.

Example:

- If one pod dies → creates another pod

Simple:

ReplicaSet ensures desired number of pods are running.

Deployment

Used to manage ReplicaSets and Pods.

Features:

- Rolling updates

- Rollback
- Scaling

  Simple:

  Deployment manages application updates and scaling.

Deployment YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx
        ports:
        - containerPort: 80
```

Create Deployment

```
kubectl apply -f deployment.yml
```

Scaling Deployment

Manual Scaling

```
kubectl scale deployment nginx-deployment --replicas=5
```

Auto Scaling

Horizontal Pod Autoscaler (HPA)

Automatically increases/decreases pods based on:

- CPU
- Memory

Simple:

HPA scales pods automatically based on load.

Create HPA

```
kubectl autoscale deployment nginx-deployment --cpu-percent=50 --min=2 --
max=5
```

Vertical Pod Autoscaler (VPA)

- Increases CPU and memory of existing pods

Simple:

VPA increases pod resources automatically.

Cluster Autoscaler

- Adds/removes worker nodes automatically

  Simple:

  Cluster Autoscaler scales nodes automatically.

Self Healing

If pod crashes:

- Kubernetes automatically recreates pod

  Simple:

  Kubernetes automatically replaces failed pods.

Kubernetes Services

Why Service?

Pods IP changes frequently.

Service provides:

- Stable IP
- Stable DNS

Types of Services

1. ClusterIP

Default service type.

Accessible:

- Inside cluster only

Simple:

ClusterIP exposes app internally.

ClusterIP YAML

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
  type: ClusterIP
```

---

2. NodePort

Exposes app outside cluster using node IP + port.

Port Range:

```
30000-32767
```

Simple:

NodePort exposes application externally through node port.

NodePort YAML

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30007
```

Access:

```
http://NodeIP:30007
```

---

3. LoadBalancer

Used mainly in cloud environments like:

- AWS
- Azure
- GCP

Creates external load balancer.

Simple:

LoadBalancer exposes application to internet.

LoadBalancer YAML

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-lb
spec:
  type: LoadBalancer
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
```

---

4. ExternalName

Maps service to external DNS.

Simple:

ExternalName redirects service to external domain.

Namespace

Used to divide cluster into multiple virtual environments.

Examples:

- dev
- test
- prod

Simple:

Namespace separates resources logically.

Create Namespace

```
kubectl create namespace dev
```

ConfigMap

Stores non-sensitive configuration data.

Examples:

- URLs
- Environment variables

Simple:

ConfigMap stores application configuration.

Secret

Stores sensitive data.

Examples:

- Passwords
- Tokens
- API keys

Simple:

Secret stores confidential information securely.

Ingress

Used to expose HTTP/HTTPS routes.

Advantages:

- Single LoadBalancer
- Domain-based routing
- SSL support

Simple:

Ingress manages external access to services.

Ingress YAML

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: nginx-ingress
spec:
  rules:
  - host: myapp.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: nginx-service
            port:
              number: 80
```

Persistent Volume (PV)

Physical storage in cluster.

Examples:

- EBS
- NFS

Simple:

PV provides storage to pods.

Persistent Volume Claim (PVC)

Request for storage.

Simple:

PVC requests storage from PV.

DaemonSet

Runs one pod on every node.

Examples:

- Monitoring agents

- Log collectors

  Simple:

  DaemonSet runs pod on all nodes.

StatefulSet

Used for stateful applications.

Examples:

- MySQL
- MongoDB

Features:

- Stable hostname
- Persistent storage

Simple:

StatefulSet manages stateful applications.

Job

Runs task once and exits.

Example:

- Backup job

  Simple:

  Job executes task one time.

CronJob

Runs jobs periodically.

Example:

- Daily backup

  Simple:

  CronJob runs scheduled jobs.

Taints and Tolerations

Used to control pod scheduling.

Simple:

Taints block pods, tolerations allow pods.

Labels and Selectors

Labels:

- Key-value pairs attached to objects

Selectors:

- Used to identify matching pods

Simple:

Labels organize resources.

Rolling Update

Updates application without downtime.

Simple:

Rolling update updates pods gradually.

Rollback

Reverts application to previous version.

Command:

```
kubectl rollout undo deployment nginx-deployment
```

Important kubectl Commands

Cluster Info

```
kubectl cluster-info
```

Get Nodes

```
kubectl get nodes
```

Get Pods

```
kubectl get pods
```

Get All Resources

```
kubectl get all
```

Describe Pod

```
kubectl describe pod pod-name
```

Logs

```
kubectl logs pod-name
```

Exec into Pod

```
kubectl exec -it pod-name -- /bin/bash
```

Delete Pod

```
kubectl delete pod pod-name
```

Kubernetes Interview Questions

1. Difference Between Deployment and ReplicaSet

Deployment	ReplicaSet
Manages ReplicaSet	Manages Pods
Supports rolling updates	No rolling updates
Supports rollback	No rollback

---

2. Difference Between Pod and Container

Pod	Container
Smallest K8s object	Application runtime
Contains containers	Runs app

---

3. Difference Between ClusterIP and NodePort

ClusterIP	NodePort
Internal access	External access
Default type	Uses node port

---

4. What Happens When Pod Crashes?

- Kubelet detects failure
- Controller recreates pod

---

5. Why Kubernetes?

Benefits:

- Auto scaling
- Self healing
- Load balancing
- Rolling updates
- High availability

Kubernetes Roadmap

Beginner

- Docker basics
- Pods
- Deployments
- Services
- YAML

Intermediate

- Ingress
- ConfigMap
- Secret

- HPA
- Volumes

Advanced

- Helm
- Monitoring
- Security
- RBAC
- CI/CD
- Service Mesh

Real-Time Interview Tips

Freshers

Focus on:

- Pod
- Deployment
- Service
- Namespace
- Scaling
- Ingress

Experienced

Focus on:

- Troubleshooting
- Autoscaling
- Security
- Monitoring
- Production issues

One-Line Important Definitions

Topic	Definition
Pod	Smallest deployable unit
Deployment	Manages pods and updates
Service	Stable network access
Namespace	Logical separation
Ingress	HTTP/HTTPS routing
ConfigMap	Stores configuration
Secret	Stores sensitive data
HPA	Auto scales pods
PV	Cluster storage
PVC	Storage request